

Project Close-out Report

Extend Open-Source Yoroi wallet to support seamless mobile Cardano dApp integration

Project name	Extend Open-Source Yoroi wallet to support seamless mobile Cardano dApp integration
Project number	1300099
Challenge	F13: Cardano Open: Developers
Project manager	Ralph Hofacker
Project start date	20 January 2025
Project completion date	31 August 2026
Budget	50,000 ADA
Repository	https://github.com/bricktowers/yoroi/tree/feature/contract-spend
Documentation	https://github.com/bricktowers/yoroi/tree/feature/contract-spend/docs/mobile-dapp-integration
Close-out video	https://youtu.be/X9bYfPBsEhQ
Examples gallery	https://github.com/bricktowers/yoroi/tree/feature/contract-spend/examples
Milestone module	https://milestones.projectcatalyst.io/projects/1300099

1. Deliverables

The project set out to make it possible for a mobile dApp to obtain a signed Cardano transaction from the Yoroi wallet running on the same device, using Universal Links on iOS and App Links on Android. All three milestones were delivered and approved.

1.1 Milestones 1 and 2 — Signing integration on iOS and Android

An application running alongside Yoroi on the same device can hand a transaction request to the wallet, have the user review and confirm it, and have the transaction signed and submitted to the chain — locking funds into a smart contract and redeeming funds from one. Delivered first on iOS, then on Android for parity. Both Proofs of Achievement were approved.

	Milestone 1 (iOS)	Milestone 2 (Android)
Demonstration	iOS demo	Android demo
On-chain evidence	b57747c1... · 1f188769...	f01fa05e... · e3c92659...
PoA approved	20 August 2025	3 September 2025

1.2 Milestone 3 — Examples and documentation

A complete developer documentation set and an examples gallery, published in the same public repository as the code.

Documentation — [docs/mobile-dapp-integration/](#), ten documents covering the integration model, a quick start, the full link reference, every field of the contract-spend request, the return path and the iOS and Android declarations it depends on, what the wallet validates and what remains the dApp's responsibility, the supported scope, building the wallet from this branch, and troubleshooting.

Illustrated with screenshots of the wallet screens taken from the demonstration recordings.

Examples gallery — `examples/README.md`, collecting every code example on one page:

- **A runnable application**, `examples/yoroi-demo` — an Expo dApp that locks ADA into a Plutus savings script with a datum on preprod, then spends that UTxO back out with a redeemer, and reads the resulting transaction hash off the return URL. This is the same application used to record the milestone 1 and milestone 2 demonstrations.
- **Four copy-paste recipes**, one per use case: lock funds into a contract, spend from a contract, request a simple ADA payment, and open a dApp in the wallet browser. Each gives the code that builds the link, the wallet screen it opens, and what comes back.
- **Simulator test scripts**, so a request can be exercised without a physical device.

1.3 What was built

The integration adds a new deep-link use case, `transfer/request/contract-spend`, to the URI schemes Yoroi already recognises (`yoroi://` and `https://yoroi-wallet.com/w1`). A dApp constructs a link describing the script inputs it wishes to spend and the outputs it wants created; the operating system routes that link to Yoroi; Yoroi builds the transaction, presents it to the user behind an explicit disclaimer screen, signs it on confirmation, submits it, and returns the user to the dApp.

The result is a general capability, not a single hard-coded flow. The request format is expressive enough to cover the interactions a Cardano dApp actually needs from a wallet:

- **Lock funds into a script**, attaching an arbitrary inline datum to the output.
- **Spend from a script**, supplying an arbitrary redeemer — so a dApp can invoke any endpoint its validator exposes, rather than a fixed set of operations.
- **Direct the proceeds anywhere**, across multiple outputs, in ADA or native tokens.
- **Leave the mechanics to the wallet** — fee estimation, collateral and change are handled by Yoroi, not by the calling dApp.

In other words, a dApp built around a Cardano script can now ask Yoroi to sign its contract interactions from a phone, with the shape of those interactions determined by the dApp's own validator rather than by the wallet.

Scope of the delivered implementation. The work targets **Plutus V3** scripts on the Cardano **preprod** network, with the script published on-chain as a reference script and inline datums. These bounds are stated in the documentation so that an integrator knows exactly what they are building against.

Environment. All development, demonstration and on-chain evidence was produced on the Cardano **preprod** network, and the transactions linked in this report can be inspected there.

Licence and availability. Apache-2.0, in a public fork of the upstream Yoroi repository, on the `feature/contract-spend` branch, based on the wallet's `production` line at v5.3.3.

2. Usage

Who this is for. Developers building mobile-first dApps on Cardano who need a user to authorise a smart-contract interaction without leaving their phone — the case that previously had no clean answer, because the desktop pattern of an injected wallet provider does not exist on mobile.

How it is used. A developer adds the `@yoroi/links` package, builds a `contract-spend` link describing the script inputs and intended outputs, and opens it. Yoroi takes over from there; the user is returned to the dApp through a `redirectTo` URL supplied in the original link. The published documentation covers the full parameter reference, the platform-side Universal Link and App Link configuration, the security constraints, and a runnable quick start. The example application in the

repository implements this end to end and is the same application used to record the milestone demonstrations.

Demonstrated usage to date. Two recorded demonstrations, one per platform, each producing verifiable preprod transactions — four in total, all linked above. The example application, the test scripts and the documentation are published so that any developer can reproduce the flow independently.

Adoption. We report no third-party adoption metrics at the time of writing. This deliverable is developer infrastructure published at the close of the project rather than a consumer-facing product, and the documentation and examples that make adoption practical are themselves part of the final milestone. What we can report is that the integration is complete, demonstrated, reproducible from a public repository, and reusable: the link schema is not specific to Yoroi, and any Cardano mobile wallet could implement the same contract-spend use case against it.

3. Impact

3.1 Before and after

Before this project, a mobile dApp had no route to have a Cardano contract interaction signed by a wallet on the same device, and contract-driven dApps were effectively browser-only. After it:

- A defined deep-link use case, implemented on both iOS and Android at parity.
- The same class of contract interactions is available to a mobile app: lock with a datum, spend with any redeemer the validator accepts, proceeds to multiple outputs in ADA or native tokens.
- Requests are validated against a strict, length-bounded schema; unknown parameters and insecure return URLs are rejected.
- The user sees an explicit disclaimer and review screen before any signature.
- Public documentation, a worked example application, integration recipes and simulator test scripts.

3.2 Challenge KPIs

The F13 *Cardano Open: Developers* challenge seeks open-source tooling and infrastructure that lowers the barrier for developers building on Cardano.

Challenge KPI	How the project addressed it
Open-source contribution to the Cardano developer ecosystem	All work published under Apache-2.0 in a public repository, with documentation and a worked example
Tooling that reduces developer friction	A documented, schema-validated integration path that removes the need for each dApp to invent its own wallet hand-off
Demonstrable, verifiable output	Two platform demonstrations and four verifiable on-chain transactions

3.3 Project KPIs

The proposal stated the goal as leveraging Universal Links on iOS and App Links on Android "to enable seamless integration between mobile dApps and wallets for a smooth, mobile-only experience", delivered across three milestones.

Project KPI	Outcome
Wallet integration developed for the first operating system	Delivered (iOS); Proof of Achievement approved 20 August 2025

Project KPI	Outcome
Wallet integration developed for the second operating system	Delivered (Android); Proof of Achievement approved 3 September 2025
Example application demonstrating wallet-dApp interaction	Delivered and published; used for both milestone demonstrations
Developer documentation and examples gallery	Delivered and published in the project repository
Transaction signed and submitted after user confirmation	Demonstrated on both platforms, evidenced by four preprod transactions

The proposal's impact statement anticipated "broader adoption and utility across the Cardano ecosystem". Adoption is a downstream outcome that cannot be measured at the point of publication, and we have not attempted to claim it. What the project delivered is the precondition for it: a working, documented, reusable integration path where none existed.

4. Sustainability

Where the code lives. The work remains in the public Brick Towers fork of Yoroi, on the `feature/contract-spend` branch, under Apache-2.0. The proposal anticipated this outcome explicitly: the intention was to offer the work to the upstream repository, with the stated contingency that if it were not taken up, the open-source fork would remain available for community developers to learn from. That is the position today, and the repository will remain public and unchanged as a permanent reference.

Forking and reuse. The branch is self-contained and buildable, based on the wallet's `production` line at v5.3.3. The documentation records the base version explicitly so that anyone reproducing the work knows exactly what they are building against. The branch is deliberately not rebased onto a newer upstream base: the delivered functionality was reviewed and approved against this base, and preserving it keeps the approved evidence and the published code in agreement.

Maintenance model. This was scoped as a fixed deliverable rather than an ongoing service, and it is complete. There is no revenue model attached: the work is published as open source for the ecosystem's benefit. Brick Towers continues to build on Cardano and adjacent networks and remains reachable for questions about the integration.

Permanent availability. Public GitHub repository, Apache-2.0 licence, with the documentation, example application and test scripts held in the same repository as the code they describe.

5. Key lessons learned

Contributing to a mature mobile wallet asks more of a team than the feature work alone suggests. The codebase spans several languages and a substantial build system, and getting the applications to build and run from the available documentation proved a long exercise in itself — it did not build against the current Xcode release, and we moved to an earlier version to obtain a compiling project. Any team planning a contribution to a wallet of this size would do well to budget for that explicitly rather than treat it as setup.

An older React Native foundation constrains what can be changed safely. The wallet is built on a mature React Native stack with correspondingly mature dependencies. This compounded the build difficulties and shaped our decision to deliver against a fixed, known-good base version rather than track a moving upstream.

"Open source" and "modifiable" are not the same thing. Parts of the transaction-building layer this project depends on — notably `@emurgo/yoroi-lib` and the `cross-csl` packages — are published to npm under open licences but without a public source repository. They can be called; they cannot be read, forked or extended. That shaped the deliverable: transaction assembly for contract spending had to be implemented inside the application rather than contributed to the library where it would naturally belong. For anyone evaluating a contribution to an open-source wallet, the availability of the *source* — not merely the licence — is worth checking first.

Taken together, these three conditions generated considerable unplanned work and materially shaped both the cost of the project and the form the integration had to take. None of them was visible from the outside at proposal time.

Small overheads accumulate in a large codebase. Adding one user-facing string to this wallet means editing 27 localisation files. There was no existing way to exercise deep links against a simulator, so we wrote one. Neither is remarkable alone; together they are a meaningful fraction of the project.

When effort overruns, protect the artefact with the longest life. These obstacles pushed the first two milestones past their planned effort. We prioritised the working, reviewed code — the part a competent engineer can pick up and reuse — and deferred the documentation and examples into the final milestone. We consider that the correct call: the code could not have been reconstructed from a report, and both earlier milestones were approved on the strength of it.

External conditions affected delivery pace. The project was resourced against a market cycle that was widely forecast to be expansionary and turned out to be the opposite. For a small company carrying an over-budget project, that materially reduced the resources available to complete the final milestone, and is the principal reason for the interval between the second milestone's approval in September 2025 and the delivery of this final one.

6. Next steps

- **Adoption of the pattern by other wallets** is where the greatest value lies. Nothing in the link schema is specific to Yoroi: it describes a contract interaction in wallet-neutral terms — inputs, redeemers, datums, targets — and the documentation explains the reasoning behind each constraint. Any Cardano wallet implementing it would immediately serve every dApp already built against it, giving mobile dApp builders one integration target rather than a separate one per wallet, and moving a further part of Cardano's application surface onto the phone. We would welcome conversations with any wallet team considering it.
- **Mobile dApps built on the pattern.** With the request format documented and a worked example published, dApp teams can build contract-driven mobile applications against it directly. That is the outcome the schema exists to serve.
- **The repository remains published** as a permanent, public reference for developers integrating mobile dApps with a Cardano wallet.
- **Upstream adoption** remains open. The work is available under Apache-2.0 for Emurgo or any other party to take up; the proposal anticipated both outcomes, and the fork stands as the fallback.

7. Final thoughts

The project delivered what it proposed, and a little more than the proposal's wording suggests. A mobile dApp can now ask the Yoroi wallet on the same device to sign and submit a Cardano contract interaction — locking funds with a datum, invoking any redeemer its validator exposes, moving ADA and native tokens — on both iOS and Android, with the user in control of every

confirmation. The whole path is documented and reproducible from a public repository, and the request format it defines belongs to no single wallet.

Getting there cost more than we planned, for reasons that were mostly not about the feature. We record them plainly because the next team attempting a wallet contribution will meet the same conditions.

We are grateful to the reviewers, whose feedback across the earlier milestones was constructive and prompt, and to the Catalyst community for funding work that makes the mobile path on Cardano a little less difficult than we found it.

8. References

- Proposal — <https://projectcatalyst.io/funds/13/cardano-open-developers/extend-open-source-yoroi-wallet-to-support-seamless-mobile-cardano-dapp-integration>
- Milestone module — <https://milestones.projectcatalyst.io/projects/1300099>
- Repository, delivery branch — <https://github.com/bricktowers/yoroi/tree/feature/contract-spend>
- Documentation — <https://github.com/bricktowers/yoroi/tree/feature/contract-spend/docs/mobile-dapp-integration>
- Examples gallery — <https://github.com/bricktowers/yoroi/tree/feature/contract-spend/examples>
- Close-out video — <https://youtu.be/X9bYfPBsEhQ>
- Milestone 1 demonstration (iOS) — <https://youtu.be/Ajpkn94gqc4>
- Milestone 2 demonstration (Android) — https://youtu.be/ygR_IZHCuag
- On-chain evidence — four preprod transactions, linked in §1.1